JavaScript Fundamentals — Day 2

Control Flow & Functions — Detailed Beginner Notes

1. Control Flow

JavaScript normally runs code from top to bottom. **Control flow** lets you make decisions, repeat code, or organize reusable logic.

Decisions: if, else if, else, switch
Repetition: for, while, do...while
Reusable logic: functions

2. if Statement

Use `if` when code should run only when a condition is true.

```
if (condition) {
   // code runs when true
}
```

A condition usually produces `true` or `false`.

```
let age = 20;
if (age >= 18) {
  console.log("Adult");
}
```

3. else

`else` gives an alternative when the `if` condition is false.

```
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

4. else if

Use `else if` when there are multiple possible conditions. JavaScript checks from top to bottom and stops at the first true condition.

```
if (marks >= 90) {
  console.log("A");
} else if (marks >= 75) {
  console.log("B");
} else if (marks >= 40) {
  console.log("C");
} else {
  console.log("F");
}
```

Key idea: once one branch is true, the remaining else-if/else branches are skipped.

5. switch Statement

Use `switch` when comparing one value with several specific possible values.

```
switch (value) {
  case 1:
    // code
```

```
    break;
  case 2:
    // code
    break;
  default:
    // no match
}
```

case checks a possible value. **break** stops the switch after a matching case. **default** runs when no case matches.

6. Loops

A loop repeats a block of code. The three loops covered today are `for`, `while`, and `do...while`.

for loop

A `for` loop is commonly used with a counter or a known repetition pattern.

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Three parts: initialization → where to start; condition → when to continue; update → how the counter changes.

Execution: Start → Check → Run → Update → Check again.

while loop

A `while` loop checks its condition before each iteration.

```
let i = 1;
while (i <= 4) {
  console.log(i);
  i++;
}
```

Important: the condition or relevant variable must eventually change, otherwise the loop can become infinite.

do...while loop

A `do...while` loop runs its body at least once and checks the condition afterward.

```
let i = 5;
do {
  console.log(i);
  i--;
} while (i > 2);
```

Remember: while → check first, run second. do...while → run first, check second.

7. Functions

A **function** is a reusable block of code designed to perform a particular task.

```
function sayHello() {
  console.log("Hello!");
}
```

Defining a function creates its instructions. **Calling** a function executes those instructions.

```
sayHello();
```

8. Function Parameters

A **parameter** is a variable in the function definition that receives information when the function is called.

```
function greet(name) {
  console.log("Hello!, " + name);
}

greet("Namratha");
```

Parameter: `name` — a placeholder. **Argument:** `"Namratha"` — the actual value passed during the call.

9. Return Values

`return` sends a value back from a function to the code that called it. This lets the result be stored or used elsewhere.

```
function multiply(a, b) {
  return a * b;
}

let result = multiply(4, 5);
console.log(result);
```

Flow: 4 and 5 enter the function → 4 × 5 becomes 20 → return sends 20 back → `result` stores 20.

console.log() displays a value. **return** sends a value back. A return statement also ends that function call.

10. Arrow Functions

An **arrow function** is an alternative, usually shorter syntax for creating a function. It uses =>.

```
const multiply = (a, b) => {
  return a * b;
};
```

When the function body is only one expression being returned, it can be shortened to:

```
const multiply = (a, b) => a * b;
```

Both forms return the multiplication result.

11. Day 2 Quick Revision

Topic	Remember
if	Run code when a condition is true.
else if	Try another condition when earlier one is false.
else	Run when none of the previous conditions are true.
switch	Compare one value with specific cases.
for	Repeat using initialization, condition and update.
while	Check the condition before running each iteration.
do...while	Run once first, then check the condition.
function	Reusable block of code.
parameter	Placeholder that receives input.
argument	Actual value passed to a function.

Topic	Remember
return	Sends a value back from a function.
arrow function	Shorter function syntax using =>.

Day 2 complete: if, else if, else, switch, for, while, do...while, functions, parameters, return values, and arrow functions.